

* Number of approaches to re-use module:-

→ we know that a module is a collection of variable, function and classes.

→ To re-use the features (variable names, function names and class names) of module, we have 2 approaches.

They are:

① By using import statements.

② By using from import statements.

① By using import statements:-

→ 'import' is a keyword

→ The purpose of import statement is that "to refer or access the variable names, function names and class names in current program".

→ we can use import statement in 4-ways

syntax ① import module name

↓
this syntax imports single module

example) #import syntax-1.py

```
import icici
```

```
import mathinfo
```

```
import aop
```

```
print("Bank Name: {0}".format(icici.bname))
```

```
print("Bank Add: {0}".format(icici.addr))
```

```
icici.simpleint()
```

```
print(" — — — ")
```

```
print("val of PI =", mathinfo.PI)
```

```
print("val of E =", mathinfo.E)
```

```
print(" — — — ")
```

```
aop.sumop(100, 200)
```

```
print(" — — — ")
```

Syntax ② import module name 1, module name 2 ... module n
↓
this syntax imports multiple modules.

ex). #import syntax-2.py
import icici, mathinfo, aop
~~from~~ Baki Sab same

Syntax ③ import module name as alias name
↓
this syntax imports single module and aliased with another unique name.

e.g). import icici as i
import mathinfo as m
import aop as a.

Bank name ke jagah = i
Add = i
i.simplint
m.PI, m.e,
a.sumop -

Syntax ④ import module name 1 as alias name, module name 2 as alias name ... module name n as alias name.
↓

This syntax import multiple modules and aliased with another unique names.

e.g) import icici as i, mathinfo as m, aop as n

② By using from... import statement:-

⇒ Here "from" "import" are the keywords.

⇒ The purpose of from... import statement is that "to refer or access the variable names, function names and class names in current program directly without writing module name as alias name of module name".

⇒ we can use from... import statement in 3 ways.

Syntax ①

from module name import Variable Names, Function Names, Class Names;

↓
• This syntax imports the Variable Names, Function Names, Class Names. of a module

e.g)

```
from icici import bname, addr, simpleint
from mathsinfo import PI, E
from aop import sumop, subop
print ("Bank Name: {}".format(bname))
print ("Bank Address: {}".format(addr))
simpleint()
print (" - - - ")
print ("val of PI =", PI)
print ("val of E =", E)
print (" - - - ")
sumop(100, 200)
subop(10, 20)
print (" - - - ")
```

Syntax ② from module name import Variable Names as alias name,
Function Names as alias name, Class Names as
alias names.

↓
• This syntax imports the variable names, function names, class names of a module with unique alias names.

e.g) from icici import bname as b, addr as a, simpleint as si
from mathsinfo import PI as P, E
from aop import sumop as ap, subop as sp
बाकी सब वैसा Bank Name के जगह - b, Add = a, PI, P, E, E,
sumop → ap
subop → sp

Syntax ① :- from module name import *

⇒ This syntax imports All variable Names, function Names, class Names of a module.

⇒ This syntax is not recommended to use bcoz it imports required features of module and also imports un-interested features also imported and leads more main memory space.

e.g) . from icici import *
from math import *
from aop import *

(Baaki lab syntax ~~श्री~~ श्री),

Q. write python program which will generate multiplication table using modules.

multable.py - - - Filename and module Name.

```
def table(n):
```

```
    if (n <= 0):
```

```
        print("{} is invalid input".format(n))
```

```
    else:
```

```
        print("multable for : {}".format(n))
```

```
        for i in range(1,11):
```

```
            print("{} x {} = {}".format(n,i,n*i))
```

multable demo.py

```
from multable import table
```

```
table(int(input("Enter a number for generating mul table:")))
```

Q. write a python program which will accept a numerical integer value and decide whether prime or not by using modules.

prime.py - - - file name & module Name

```
def decideprime(n):  
    if (n <= 1):  
        print("{} is invalid".format(n))  
    else:  
        res = "PRIME"  
        for i in range(2, n):  
            if (n % i == 0):  
                res = "NOT PRIME"  
                break  
        if (res == "PRIME"):  
            print("{} is {}".format(n, res))  
        else:  
            print("{} is {}".format(n, res))
```

primeDemo.py

```
from prime import decideprime  
decideprime(int(input("Enter any number to decide prime  
or not:")))
```

Day 5

* Reloading a module in python:-

=> To reload a module in python, we use a predefined function. called `reload()`, which is present in `imp` module and it was deprecated in favour of `importlib` module.

syntax : `imp.reload(module name)`
 (or)

`importlib.reload(module name)` ---> recommended

=> purpose/situation:-

- `reload()` reloads a previously imported module.
- If we have edited the module source file by using an external

editor and we want to use the changed values/ updated values / new versions of previously loaded module then we use reload().

example.

shares.py - - - - - filename & module Name.

```
def sharesinfo():  
    d = {"Tf": 5, "Fin": 4, "Auto": 3, "pharma": 2}  
    return d.
```

sharesDemo.py

```
import shares, time, importlib
```

```
def dispshares(d):  
    print(" — — — — —")  
    print("\tshare Name\tshare value")  
    print(" — — — — —")  
    for sn, sv in d.items():  
        print("\t{<3}\t{<3}".format(sn, sv))  
    print(" — — — — —")
```

main program

```
d = shares.sharesinfo()  
dispshares(d)  
print("I am going to sleep for 20 secmd")  
time.sleep(20)  
print("I am coming out of sleep")
```

reload the module

```
importlib.reload(shares)  
d = shares.sharesinfo()  
dispshares(d)  
print("I am going to sleep for 20 secmd")  
time.sleep(20)  
print("I am coming out of sleep")
```

reload the module

```
importlib.reload(shares)  
d = shares.sharesinfo()      dispshares(d) =
```

Case studies ①

Q. ① write a python program which will implement the following by using modules.

Arithmetic operations

1. Addition
2. subtraction
3. multiplication
4. Normal Division
5. Floor Division
6. modulo division
7. exponentiation
8. Exit.

File system

1. Topmenu.py - - - - Filename and module name
2. TopImplementations.py - - - - File name & module Name
3. TopDemo.py - - - - main program

#Appmenu.py

```
def menu():
```

$s = \text{""} \backslash t \backslash t = \text{--- -- -- -- --}$

Asthetic operation

Same as it is

```
print(s).
```

Top implementations - py. file name & module name

```
def addop():
```

```
print ("Enter two values for Addition")
```

a, b = int(input()), int(input())

```
print("sum({}, {}) = {}".format(a,b, a+b))
```

येसा सब से बडी operation subtraction, multiply, division etc

```

# Aop Demo.py
from Aopmenu import menu
from AopImplementations import *
while (True):
    menu()
    ch = int(input("Enter ur choice?"))
    match (ch):
        case 1: addop()
        case 2: subop()
        case 3: mulop()
        case 4: divop()
        case 5: fdivop()
        case 6: modop()
        case 7: expop()
        case 8:
            print("Thx for using this program")
            break
        case _:
            print("ur selection of operations is wrong!!")

```

Case studies ②

Q ②. write a python program which will implement the following by using modules.

Different figures

1. Circle Area
2. Circle perimeter
3. Square Area
4. Square perimeter
5. rectangle Area
6. rectangle Perimeter
7. Triangle Area
8. Triangle Perimeter ($p = a + b + c$)
9. Exit

File system

circle.py Filename & module name
 area()
 Perim()

Square.py - - - File name & module name

area()

peri()

rectangle.py - - -

area()

peri()

Triangle.py - - -

area()

peri()

Figuremenu.py - - -

CSRT.py - - - main program =

→ # circle.py

def area():

r = float(input("Enter radius of Area:"))

a = $3.14 * r * r$

print("Area of circle = {}".format(a))

def peri():

r = float(input("Enter radius of Peri:"))

p = $2 * 3.14 * r$

print("Perimeter of circle = {}".format(p))

square.py.

area of square = $(side)^2$

perimeter of square = $4 * side$

rectangle.py.

Area of rectangle = $l * b$. (la)

perimeter of rectang = $2 * (l + b)$.

Figuremenu.py

def menu:

s = 1 to different figure as it is

print(s)

#CSRT.PY - - - main program

from figure menu import menu

from circle import area as ca, peri as cp

from square import area as sa, peri as sp

from rect import area as ra, peri as rp

while (True):

 menu()

 ch = int(input("Enter ur choice:"))

 if (ch == 1):

 ca()

 elif (ch == 2):

 cp()

 elif (ch == 3):

 sa()

 elif (ch == 4):

 sp()

 elif (ch == 5):

 ra()

 elif (ch == 6):

 rp()

 elif (ch == 7):

 print("student must do")

 elif (ch == 8):

 print("student must do")

 elif (ch == 9):

 break

 else:

 print("ur selection of operation is wrong")

==